ОСНОВНОЙ ЭКРАН РОБОТА (КИОСК): ДВИГАТЕЛИ, ГРУЗОВОЙ ОТСЕК, АКБ

> **Документ:** docs/MAIN_SCREEN.md **Файлы:** gui/main.html, gui/main.css, gui/main.js, gui/backend.py, gui/serve.py **Версия:** 2.0 (визуал + бэкенд) · **Дата:** 2026-10 **Связанные документы:** RUN_AND_ACCESS.md (как попадать на экраны) · GUI.md · SERVICE_CONSOLE.md · SERIAL_PROTOCOL.md · SOFTWARE_ARCHITECTURE.md

Это экран, который видят на самом роботе: открывается автоматически при запуске сервера на борту и не требует прокрутки, клавиатуры и мыши — только касания.

---

1. Запуск на роботе

Сервер борта **окон не открывает**: он отдаёт страницы и данные любому браузеру. Основной экран — статичная страница на дисплее робота: там обновляются показания и вводят PIN-код, которым открывается отсек на самом роботе. Инженерный пульт открывают удалённо (/console).

```
python3 gui/serve.py                      # == python3 gui/backend.py: только сервер
python3 gui/serve.py --source serial --ports /dev/ttyUSB0,/dev/ttyUSB1,/dev/ttyUSB2,/dev/ttyUSB3
python3 gui/serve.py --source ros         # данные из ROS 2 (rclpy)
python3 gui/serve.py --pin 2580 --reset-pin
python3 gui/serve.py --kiosk              # разово: + браузер без рамок на дисплее робота
python3 gui/serve.py --open               # разово: + страница в обычном окне браузера
```

Что делает --source:

Значение	Источник	Когда использовать
sim (по умолчанию)	демонстрационная физика внутри бэкенда	стенд, показ, отладка экрана без робота
serial	4 UART, кадры телеметрии 16 Б (CRC16/Modbus), нужен pyserial	реальные модули напрямую с бортового ПК
ros / auto	ROS 2: /screen/state, /battery/voltage, /battery/current, /screen/motors	штатная работа через ROS 2; при отсутствии rclpy автоматически падает в sim

Адреса печатаются в консоли: http://127.0.0.1:8080/main.html (сам робот) и http://<ip-робота>:8080/main.html (планшет рядом). Кэш отключён (Cache-Control: no-store) — после обновления файлов достаточно перезагрузить страницу.

Автозапуск — два независимых сервиса systemd (готовые файлы в deploy/, порядок установки — deploy/README.md):

Сервис	Что делает
rus-slam-server.service	поднимает gui/backend.py (сервер, окон не открывает)
rus-slam-display.service	показывает статичную страницу на дисплее робота: ждёт сервер и открывает http://127.0.0.1:8080/ в браузере без рамок

Если у робота нет дисплея — достаточно первого сервиса: страницы открывают с планшета или ноутбука, пульт — удалённо по /console.

---

2. Компоновка экрана

```
┌──────────────────────────────────────────────────────────────────────────┐
│ RUS SLAM · робот-курьер        АВТОНОМНЫЙ РЕЖИМ      ● модули на связи  12:41 │
├──────────────────────────────────────────────────────────────────────────┤
│ ДВИГАТЕЛИ                      │ АКБ                 │ ГРУЗОВОЙ ОТСЕК        │
│                                │                     │  ЗАКРЫТО              │
│ ┌──────────┐ ┌──────────┐      │     ┌─────────┐     │      • • • •          │
│ │ FL  готов │ │ FR  готов │      │     │  78 %   │     │                       │
│ │ ◔ 0°     │ │ ◔ 0°     │      │     └─────────┘     │   ┌───┬───┬───┐       │
│ │ 0 об/мин │ │ 0 об/мин │      │ 41,2 В │ 12,4 А     │   │ 1 │ 2 │ 3 │       │
│ └──────────┘ └──────────┘      │ 14 км  │ разряд     │   ├───┼───┼───┤       │
│                                │ порог 35,5 В        │   │ 4 │ 5 │ 6 │       │
│ ┌──────────┐ ┌──────────┐      │                     │   ├───┼───┼───┤       │
│ │ RL  готов │ │ RR  готов │      │                     │   │ 7 │ 8 │ 9 │       │
│ └──────────┘ └──────────┘      │                     │   ├───┼───┼───┤       │
│ в движении: 0 из 4             │                     │   │C  │ 0 │ ⌫ │       │
│                                │                     │  [ Открыть отсек ]    │
├──────────────────────────────────────────────────────────────────────────┤
│ маршрут: склад → зона выгрузки   тяга: 2,4 кВт · КПД 0,86   инженерный пульт │
└──────────────────────────────────────────────────────────────────────────┘
```

Блоки:

Блок	Что показывает	Крупные элементы
Двигатели	4 модуля (FL/FR/RL/RR): угол руля на циферблате, об/мин, температура, полоса нагрузки, чип состояния (готов / движение / хоминг / перегрев)	цифры ≥ 21 px, циферблат ≤ 130 px
АКБ	Заряд **в процентах** кольцом (центр — число), напряжение, ток, запас хода, состояние, подсказка о порогах 35,5 / 33,5 В	% — до 64 px
Грузовой отсек	Состояние замка (ЗАКРЫТО / ОТКРЫТО / БЛОКИРОВКА), маска PIN, сенсорная клавиатура 0–9/⌫/СБРОС, одна кнопка действия («Открыть отсек» ↔ «Закрыть отсек»), масса груза	клавиши ≥ 44 px, PIN — до 40 px

Шапка: название, режим, индикатор связи с модулями, часы. Подвал: маршрут, оценка мощности тяги, подсказка про инженерный пульт.

---

3. Что уже работает (визуальная часть)

- сенсорная клавиатура: ввод PIN точками, «СБРОС», подтверждение «Открыть отсек»;
- проверка PIN 2580 (демо), сообщение об ошибке, **5 неудач → блокировка 30 с** с обратным отсчётом и запретом кнопки;
- перевод состояния замка в ОТКРЫТО и отметку «груз доступен»;
- живые показания двигателей (угол, об/мин, температура, чип состояния);
- кольцо заряда с цветовой индикацией: норма → низкий заряд → аварийный;
- часы, индикатор связи, счётчик «в движении: N из 4».

Числа сейчас демонстрационные (плавно меняются), чтобы согласовать вид.

2.1. Правило: одно действие — один орган управления

Экран рассчитан на человека в перчатках и на очередь у борта, поэтому в интерфейсе нет дублирующих элементов — каждое действие выполняет ровно один орган управления, и каждое состояние показывается в одном месте.

Действие или состояние	Единственный орган управления / место
Открыть отсек	кнопка действия (при закрытом отсеке надпись «Открыть отсек»); нужен PIN

Действие или состояние	Единственный орган управления / место
Закрыть отсек	та же кнопка при открытом отсеке (надпись «Закрыть отсек»)
Стереть введённый PIN	клавиша **СБРОС** на клавиатуре (второй кнопки сброса нет)
Удалить одну цифру	клавиша ⌫
Состояние замка	чип в заголовке панели «Грузовой отсек»
Состояние груза	строка под клавиатурой («закреплён и заперт» / «доступен для погрузки/выгрузки»)
Заряд АКБ в процентах	кольцо АКБ (подпись под числом — уровень: НОРМА / НИЗКИЙ / КРИТИЧЕСКИЙ)
Ток, напряжение, запас хода, режим	строки панели «Аккумулятор» (дублируются только на пульте, не на экране)
Связь с модулями	подвал панели «Двигатели»: «в движении: X из 4 · модули на связи: Y из 4»
Канал данных (сервер/демо)	чип в шапке экрана
Последний доступ	строка «журнал: …» в подвале экрана
Переход на инженерный пульт	ссылка «инженерный пульт →» в подвале (одна, высота ≥ 44 px)

Сообщение под PIN-кодом — только отклик на последнее действие («Доступ разрешён», «Неверный PIN. Осталось попыток: N», «Ввод заблокирован…»), оно не повторяет состояние замка, которое уже показывает чип. Соблюдение правила проверяется автоматически, см. §6.

---

4. Бэкенд (`gui/backend.py`)

Один процесс отдаёт статику обоих экранов, JSON API и работает с замком. Без внешних зависимостей (кроме `pyserial` для `--source serial` и ROS 2 для `--source ros`).

4.1. HTTP API

Метод и путь	Назначение	Ответ
`GET /api/state`	снимок состояния (экран опрашивает 300 мс)	`{ok, data{…}}`
`POST /api/lock/open`	открыть отсек по `{"pin":"…"}`	`{ok:true, lock{}}` или 403 `{ok:false, reason, attemptsLeft}`
`POST /api/lock/close`	закрыть отсек	`{ok:true, lock{}, hardware}`
`POST /api/lock/pin`	сменить PIN: `{"current":"…","new":"…"}`	`{ok:true}` или 403 `{ok:false, detail}`
`GET /api/audit?limit=N`	журнал доступа и целостность цепочки	`{ok, audit[], integrity{}}`
`GET /api/health`	живость, источник данных, время работы	`{ok, source, uptimeSec}`
`GET /` · `GET /console`	302: на основной экран · на инженерный пульт	—

Состав data в `/api/state`:

```
motors[] : id, title, angle (град), rpm, temp (°C), homed, online, fault
battery  : soc (%), volts, amps, watts, remainingWh, rangeKm, state, level
cargo    : kg, closed
lock     : open, blocked, remainingMs, attemptsLeft, changedAt
прочее   : mode, route, powerKw, linkOk, source, uptimeSec, ts
```

Примеры запросов:

```
curl -s http://127.0.0.1:8080/api/state | python3 -m json.tool | head -20
curl -s -X POST http://127.0.0.1:8080/api/lock/open \
     -H 'Content-Type: application/json' -d '{"pin":"2580"}'
```

4.2. Данные экрана: откуда берутся

Данные	sim	serial (UART)	ros
Угол, об/мин, состояние модуля	модель	поля кадра 16 Б	/screen/motors
Напряжение и ток	модель	усреднение по 4 модулям	/battery/*
Заряд в процентах	по расходу энергии	по OCV-кривой 12S3P	по OCV-кривой
Запас хода	714,2 Вт·ч ÷ расход	то же	то же
Уровень заряда	пороги 35,5 / 33,5 В	то же	то же
Груз, маршрут, режим	константы демо	состояние	/screen/state

Пороговые значения и OCV-кривая совпадают с gui/console-core.js (RS.pack) — экран и сервисный пульт показывают одинаковые цифры.

4.3. Замок грузового отсека

- PIN: SHA-256(salt + PIN), файл gui/state/lock.json (соль, хеш, счётчик попыток, время блокировки, журнал аудита с хеш-цепочкой);
- замок на роботе один: и основной экран, и инженерный пульт (когда он открыт через тот же сервер) работают с этим хранилищем по API, поэтому PIN и журнал у них общие. Подробно о PIN и способах его установки — docs/RUN_AND_ACCESS.md §3;
- 5 неудачных попыток → блокировка 30 с (настраивается --max-attempts, --lock-sec), проверка блокировки — на сервере, экран только показывает;
- реальное железо подключается переменными окружения:

```
# открыть отсек: включить реле/соленоид на 2 с
export RUS_SLAM_LOCK_CMD='gpio write 7 1 && sleep 2 && gpio write 7 0'
# закрыть отсек
export RUS_SLAM_LOCK_CMD_CLOSE='gpio write 7 0'
python3 gui/backend.py
```

Если переменные не заданы — замок программный (hardware:"software"), а экран и API работают полностью, что удобно для стенда и демонстрации.

4.4. Что ещё предстоит подключить

1. фактические датчики массы груза и температуры модулей (сейчас в кадрах телеметрии их нет — поля заполняются моделью);
2. публикацию /screen/* из mission_manager и module_bridge, чтобы --source ros работал «из коробки» на роботе;
3. аварийную индикацию: красная подсветка блока и звук при E-STOP, потере связи и напряжении < 33,5 В;
4. передачу событий экрана (открытия отсека, ошибки) в статистику смены RS.StatsStore для отчёта.

---

5. Отношение к инженерному пульту

Страница	Назначение	Открывается
main.html	основной экран: статично на дисплее робота, там же вводят PIN, отсек открывается на роботе	корень /, сервис rus-slam-display
index.html	инженерный пульт: карта SLAM, миссия, сервисное окно, статистика	/index.html, /console

Обе страницы независимы: сбой одной не влияет на другую, общие файлы — только `main.css`/`main.js` и `console-*` для пульта.

6. Проверка

```
python3 gui/backend.py --port 8081                        # вручную в браузере
python3 gui/tests/backend.test.py                         # 40 проверок бэкенда и API
NODE_PATH=/tmp/harness/node_modules node gui/tests/main.screen.test.js   # 38 проверок экрана
NODE_PATH=/tmp/harness/node_modules node gui/tests/ui.audit.test.js      # 20 проверок интерфейса
NODE_PATH=/tmp/harness/node_modules node gui/tests/main.api.test.js      # 20 сквозных (экран + API)
NODE_PATH=/tmp/harness/node_modules node gui/tests/console.api.test.js   # 17 сквозных (пульт + борт)
```

- `backend.test.py` — поведение сервера (без флагов окон не открывает, `--kiosk` и `--open` включают показ страницы явно), CRC16 и кадры протокола, разбор потока UART с мусором, модель АКБ (пороги, OCV), PIN-хранилище (попытки, блокировка, смена PIN, целостность аудита, персистентность), живой HTTP API (статика, `/api/state`, открытие/закрытие замка, журнал, смена PIN);
- `main.screen.test.js` — разметка, живые показания, кольцо АКБ в процентах, ввод PIN, открытие/закрытие, блокировка после пяти неудач, режим «демо» при отсутствии сервера, ссылка на инженерный пульт, а также отсутствие дублей (одна кнопка отсека, одна клавиша сброса, неповторяющиеся надписи и id);
- `ui.audit.test.js` — аудит разметки обеих страниц: уникальность `id`, ни одной пары органов управления с одинаковой надписью в пределах одной вкладки, существование всех `id`, на которые ссылается скрипт, отсутствие мёртвых ветвей кода (`&& false`, `if (false)`, `TODO`);
- `main.api.test.js` — сквозной прогон «экран ↔ бэкенд»: живой `backend.py`, данные `/api/state` на экране, открытие отсека по PIN через API, аудит, закрытие и блокировка после пяти неудач (проверка считает сервер);
- `console.api.test.js` — сквозной прогон «инженерный пульт ↔ борт»: PIN с пульта открывает отсек на сервере, открытие с основного экрана подхватывается пультом, смена PIN с пульта действует на борту (старый PIN перестаёт работать), журнал пульта — это журнал борта.

Всего по проекту GUI: **260 проверок** — 32 (ядро сервисного пульта) + 93 (пульт в jsdom) + 40 (бэкенд и API) + 38 (экран) + 20 (аудит интерфейса) + 20 (экран + API) + 17 (пульт + борт).

Как попадать на экраны (дисплей робота, планшет, пульт удалённо) — `docs/RUN_AND_ACCESS.md`.

7. Лицензия и авторы

MIT, команда RUS SLAM, НТЦ АО «АВТОВАЗ», конкурс «Первые в профессии», задание № 2.